

Data Preprocessing for Data Science

This notebook demonstrates common **data preprocessing techniques** used in data science workflows.

```
In [2]: # pip install scikit-learn
```

```
In [3]: # Import required libraries
import pandas as pd
import numpy as np
```

Load Dataset

In this example we use the **tips dataset** often used for demonstration. Replace the file path with your own dataset when using this notebook.

```
In [4]: # Load dataset
df = pd.read_csv('../data/tips.csv')
df.head()
```

```
Out[4]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

Normalization (Min-Max Scaling)

Normalization rescales values to the range **[0,1]**.

Useful when:

- features have very different scales
- using distance-based algorithms
- comparing variables directly
- combining variables into a score

Don't use when:

- using algorithms that are scale-invariant, such as Trees.

```
In [5]: # Apply MinMax normalization to 'total_bill'

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df['total_bill_normalized'] = scaler.fit_transform(df[['total_bill']])
df.head()
```

```
Out[5]:
```

	total_bill	tip	sex	smoker	day	time	size	total_bill_normalized
0	16.99	1.01	Female	No	Sun	Dinner	2	0.291579
1	10.34	1.66	Male	No	Sun	Dinner	3	0.152283
2	21.01	3.50	Male	No	Sun	Dinner	3	0.375786
3	23.68	3.31	Male	No	Sun	Dinner	2	0.431713
4	24.59	3.61	Female	No	Sun	Dinner	4	0.450775

Standardization (Z-score Scaling)

Standardization transforms data so that:

- Mean = 0
- Standard deviation = 1

Useful when:

- preserving distribution shape
- keeping outliers visible
- making variables comparable
- improving machine learning algorithms (Many algorithms assume features are centered around 0 and similarly scaled.)

Often used in:

- linear models
- PCA
- clustering

Don't use when:

- using algorithms that are scale-invariant, such as Trees.

```
In [6]: # Apply standardization

from sklearn.preprocessing import StandardScaler

standard_scaler = StandardScaler()
df['total_bill_standardized'] = standard_scaler.fit_transform(df[['total_bill']])
df.head()
```

Out[6]:

	total_bill	tip	sex	smoker	day	time	size	total_bill_normalized	tot
0	16.99	1.01	Female	No	Sun	Dinner	2	0.291579	
1	10.34	1.66	Male	No	Sun	Dinner	3	0.152283	
2	21.01	3.50	Male	No	Sun	Dinner	3	0.375786	
3	23.68	3.31	Male	No	Sun	Dinner	2	0.431713	
4	24.59	3.61	Female	No	Sun	Dinner	4	0.450775	

Outlier Detection and Removal (Z-score filtering)

Useful when:

- outliers strongly distort statistical measures
- using ML models sensitive to extreme values (such as: Linear/Logistic regression, K-means clustering, PCA, KNN, SVM)

Don't use when

- they represent important real events.

```
In [7]: from scipy import stats
tmpdf=df[['total_bill','tip']]
# Keep rows where all features have z-score < 3
tmpdf[(np.abs(stats.zscore(tmpdf)) < 3).all(axis=1)]
```

Out[7]:

	total_bill	tip
0	16.99	1.01
1	10.34	1.66
2	21.01	3.50
3	23.68	3.31
4	24.59	3.61
...	...	...
239	29.03	5.92
240	27.18	2.00
241	22.67	2.00
242	17.82	1.75
243	18.78	3.00

239 rows × 2 columns

Label Encoding (categorical values → numeric labels)

Example:

Sun -> 0
Sat -> 1
Thur -> 2
Fri -> 3

Useful when:

- categories have a natural order (The numbers reflect the ranking), such as: Education Levels
- For target variables (class labels) in ML models

Don't use when

- for nominal variables (no natural order or ordering is meaningless), such as: cities

```
In [9]: # Encode categorical column 'day'
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()
df['day_encoded'] = label_encoder.fit_transform(df['day'])
df.head()
```

```
Out[9]:
```

	total_bill	tip	sex	smoker	day	time	size	total_bill_normalized	tot
0	16.99	1.01	Female	No	Sun	Dinner	2	0.291579	
1	10.34	1.66	Male	No	Sun	Dinner	3	0.152283	
2	21.01	3.50	Male	No	Sun	Dinner	3	0.375786	
3	23.68	3.31	Male	No	Sun	Dinner	2	0.431713	
4	24.59	3.61	Female	No	Sun	Dinner	4	0.450775	

Label Binarization

Example:

Day	Fri	Sat	Sun	Thur
Fri	1	0	0	0

Useful when:

- categorical values should not have ordinal meaning.

- multi-class classification
- multi-label classification (instance can belong to multiple classes at the same time).

```
In [13]: # Apply label binarizer

from sklearn.preprocessing import LabelBinarizer

lb = LabelBinarizer()
binary_labels = lb.fit_transform(df['day'])
binary_df = pd.DataFrame( binary_labels, columns=lb.classes_)
binary_df.tail()
```

```
Out[13]:
```

	Fri	Sat	Sun	Thur
239	0	1	0	0
240	0	1	0	0
241	0	1	0	0
242	0	1	0	0
243	0	0	0	1

```
In [14]: df.tail()
```

```
Out[14]:
```

	total_bill	tip	sex	smoker	day	time	size	total_bill_normalized
239	29.03	5.92	Male	No	Sat	Dinner	3	0.543779
240	27.18	2.00	Female	Yes	Sat	Dinner	2	0.505027
241	22.67	2.00	Male	Yes	Sat	Dinner	2	0.410557
242	17.82	1.75	Male	No	Sat	Dinner	2	0.308965
243	18.78	3.00	Female	No	Thur	Dinner	2	0.329074

```
In [15]: pd.concat([df, binary_df], axis=1)
```

Out[15]:	total_bill	tip	sex	smoker	day	time	size	total_bill_normalized
0	16.99	1.01	Female	No	Sun	Dinner	2	0.291579
1	10.34	1.66	Male	No	Sun	Dinner	3	0.152283
2	21.01	3.50	Male	No	Sun	Dinner	3	0.375786
3	23.68	3.31	Male	No	Sun	Dinner	2	0.431713
4	24.59	3.61	Female	No	Sun	Dinner	4	0.450775
...	...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3	0.543779
240	27.18	2.00	Female	Yes	Sat	Dinner	2	0.505027
241	22.67	2.00	Male	Yes	Sat	Dinner	2	0.410557
242	17.82	1.75	Male	No	Sat	Dinner	2	0.308965
243	18.78	3.00	Female	No	Thur	Dinner	2	0.329074

244 rows × 14 columns

Data Binning (Discretization) (continuous variables → categorical bins)

Example:

- Income → Low / Medium / High

Useful when:

- want to simplify data
- want to Reduce noise
- rule-based models (Handles non-linear relationships)
- interpretability and visualization (grouped charts using bins)

Strategy	Idea	Best For
uniform	equal width bins	evenly distributed data
quantile	equal number of samples	skewed data

| Encode Option | Output Representation | Type | Notes | |---|---|---|---| | **ordinal** |
 bins represented as integers (0, 1, 2, ...) | numeric labels | | **onehot-dense** |
 one-hot vectors like [1,0,0], [0,1,0] | dense array | Stores all values, including zeros

```
In [17]: # Create bins for tip amount
from sklearn.preprocessing import KBinsDiscretizer
```

```
discretizer = KBinsDiscretizer( n_bins=3, encode='ordinal', strategy='uniform')
df['tip_bin'] = discretizer.fit_transform(df[['tip']])
df.head(20)
```

```
Out[17]:
```

	total_bill	tip	sex	smoker	day	time	size	total_bill_normalized	to
0	16.99	1.01	Female	No	Sun	Dinner	2	0.291579	
1	10.34	1.66	Male	No	Sun	Dinner	3	0.152283	
2	21.01	3.50	Male	No	Sun	Dinner	3	0.375786	
3	23.68	3.31	Male	No	Sun	Dinner	2	0.431713	
4	24.59	3.61	Female	No	Sun	Dinner	4	0.450775	
5	25.29	4.71	Male	No	Sun	Dinner	4	0.465438	
6	8.77	2.00	Male	No	Sun	Dinner	2	0.119397	
7	26.88	3.12	Male	No	Sun	Dinner	4	0.498743	
8	15.04	1.96	Male	No	Sun	Dinner	2	0.250733	
9	14.78	3.23	Male	No	Sun	Dinner	2	0.245287	
10	10.27	1.71	Male	No	Sun	Dinner	2	0.150817	
11	35.26	5.00	Female	No	Sun	Dinner	4	0.674277	
12	15.42	1.57	Male	No	Sun	Dinner	2	0.258693	
13	18.43	3.00	Male	No	Sun	Dinner	4	0.321743	
14	14.83	3.02	Female	No	Sun	Dinner	2	0.246334	
15	21.58	3.92	Male	No	Sun	Dinner	2	0.387725	
16	10.33	1.67	Female	No	Sun	Dinner	3	0.152074	
17	16.29	3.71	Male	No	Sun	Dinner	3	0.276917	
18	16.97	3.50	Female	No	Sun	Dinner	3	0.291160	
19	20.65	3.35	Male	No	Sat	Dinner	3	0.368245	

```
In [19]: df['tip_bin']=df['tip_bin'].replace(0, 'low').replace(1,'medium').replace(2,
```

```
In [20]: df
```

Out[20]:

	total_bill	tip	sex	smoker	day	time	size	total_bill_normalized
0	16.99	1.01	Female	No	Sun	Dinner	2	0.291579
1	10.34	1.66	Male	No	Sun	Dinner	3	0.152283
2	21.01	3.50	Male	No	Sun	Dinner	3	0.375786
3	23.68	3.31	Male	No	Sun	Dinner	2	0.431713
4	24.59	3.61	Female	No	Sun	Dinner	4	0.450775
...	...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3	0.543779
240	27.18	2.00	Female	Yes	Sat	Dinner	2	0.505027
241	22.67	2.00	Male	Yes	Sat	Dinner	2	0.410557
242	17.82	1.75	Male	No	Sat	Dinner	2	0.308965
243	18.78	3.00	Female	No	Thur	Dinner	2	0.329074

244 rows × 11 columns

In []: